

AI Disclosure and Verification State

- **Model:** Gemini (Google)
- **Verification State:** Verified open-source technical specification release from marielandryspyshop.com.
- **Keywords:**

AI Studio

GitHub Pages

Vite deployment

React CI/CD

automation pipeline

environment secrets

AI Studio to GitHub Pages Deployment Pipeline

- **Publisher:** Marie Landry Spy Shop (marielandryspyshop.com)
- **Access Type:** Open Source / Public Open-Access Repository

ABSTRACT

This technical specification outlines a standardized, production-ready continuous integration and continuous deployment (CI/CD) pipeline for transitioning Single-Page Applications (SPAs) generated via Google AI Studio into static hosting environments on GitHub Pages. It resolves critical structural deployment challenges in Vite and React applications, including nested repository asset path resolutions, headless GitHub Actions authentication, secure environment variable injection (such as `VITE_GEMINI_API_KEY`), and client-side routing fallback strategies. This protocol enables deterministic, zero-touch automated deployments for AI-generated web prototypes.

MASTER AI STUDIO DIRECTIVE

When ready to deploy, upload this PDF/document directly into AI Studio and execute the following master prompt:

```
"prepare the app for github deployment following the instructions in this pdf"
```

1. Scope & Objective

This technical specification standardizes the automated transition of web applications generated via AI Studio into live production environments on GitHub Pages. By standardizing Vite configurations, continuous integration workflows, and secure environment variable injection, developers can eliminate manual build errors and runtime path failures.

2. Technical Implementation Architecture

Step 1: Configure Vite for Relative Asset Paths

- Vite defaults to an absolute root path (/). When deploying to nested GitHub Pages project repositories (`https://username.github.io/repo-name/`), asset lookups fail unless the `base` property is adjusted.
- **Action:** Update `vite.config.ts` to set `base: './'` and define the production output directory:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  base: './', // CRITICAL for GitHub Pages sub-path hosting
  build: {
    outDir: 'dist',
  },
});
```

Step 2: Establish the GitHub Actions Deployment Workflow

- Create a workflow configuration file at `.github/workflows/deploy.yml`.

- **Triggers & Permissions:** Configure the runner to activate on pushes to `main` or `master` branches, alongside manual dispatching, granting explicit repository permissions (`contents: read`, `pages: write`, `id-token: write`).
- **Build and Secret Injection Sequence:** Check out code using `actions/checkout@v4`, spin up Node.js 20 via `actions/setup-node@v4`, install dependencies, and inject API secrets directly into the build command:

```
name: Deploy static content to Pages

on:
  push:
    branches: ['main', 'master']
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: 'pages'
  cancel-in-progress: true

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
      - name: Install dependencies
        run: npm install
      - name: Build static production bundle
        env:
          VITE_GEMINI_API_KEY: ${ secrets.VITE_GEMINI_API_KEY }
        run: npm run build
      - name: Setup Pages
        uses: actions/configure-pages@v5
      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: './dist'
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```